IBM Innovation Centre for Education

NCS NihonCommunicationSolutions

Yenepoya Institute of Arts, Science, Commerce, and Management

Project – Low Level Design

on

Cloud Forensics Automation for E-commerce Breaches

Student Name: Ardra P V

Industry Mentor: Mr. Shashank

Guided by: Ms. Suzana Sharol Prabhakar

Table of Contents

1. Introduction

The Cloud Forensics Automation System is designed to automate the analysis of file metadata to detect potential tampering or suspicious modifications. This Low-Level Design (LLD) document provides a detailed description of system components, function-level logic, data structures, and internal interactions between modules.

1.1 Scope of the Document

In Scope:

- Detailed module-level design
- Function-level logic and workflows
- Internal data handling mechanisms
- API-level details

Out of Scope:

- UI design styling
- Deployment infrastructure
- External integrations (none used

1.2 Intended Audience

- Developers – for implementation
- System Designers – for internal structure validation
- Evaluators – for technical assessment

1.3 System Overview

The system processes uploaded files, extracts metadata, detects suspicious modifications, and presents results through a dashboard. It operates as a single-user forensic analysis tool using a Flask backend and local storage.

2. System Design

2.1 Application Design

The system follows a **modular monolithic architecture**, where each module performs a specific task:

- File Handling Module
- Metadata Extraction Module
- Detection Engine
- CSV Storage Manager
- Dashboard Renderer

Each module communicates internally through function calls.

2.2 Process Flow

1. User uploads file(s)
2. Backend saves file securely
3. Metadata is extracted
4. Detection logic is applied
5. Results stored in CSV
6. Dashboard displays results

2.3 Information Flow

Input:

- Uploaded files

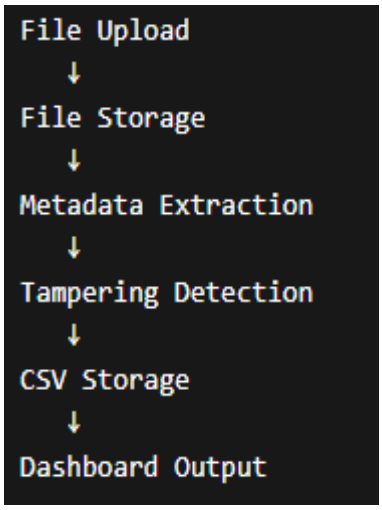
Processing:

- Extract timestamps (Creation time ctime,modified time - mtime)
- Calculate file size
- Apply tampering logic

Output:

- CSV report
- Dashboard table with suspicious flags

Figure: Information Flow of File Analysis System



2.4 Components Design

1. File Handling Module

- Handles file upload and storage
- Uses secure filename sanitization

2. Metadata Extraction Module

- Extracts:
  - Creation time
  - Modified time
  - File size

3. Detection Engine

- Compares timestamps
- Flags suspicious files

4. CSV Manager

- Writes structured results
- Reads data for dashboard

5. Dashboard Module

- Displays results
- Highlights suspicious entries

2.5 Key Design Considerations

- Simplicity over scalability
- Local storage for privacy
- Fast execution for small datasets
- No external dependencies

2.6 API Catalog

Endpoint	Method	Description
`/`	GET	Load file upload interface
`/upload`	POST	Upload and analyze files
`/dashboard`	GET	Display analysis results
`/clear`	POST	Delete all uploaded data
`/download-report`	GET	Download analysis report (CSV)

3. Data Design

3.1 Data Model

File Record:

- filename (string)
- created_time (datetime)
- modified_time (datetime)
- file_size (int)
- suspicious_flag (boolean)

3.2 Data Access Mechanism

- Uses file-based storage (CSV)
- Read/Write using Python csv module
- No ORM used

3.3 Data Retention Policies

- Data stored locally in CSV file
- Retained until manually deleted by user
- No automatic cleanup mechanism

3.4 Data Migration

- Not applicable (no database used)

4. Interfaces

External Interfaces

- None (fully local system)

Internal Interfaces

Caller	Callee	Purpose
Upload Route	File Handler	Save file
File Handler	Metadata Extractor	Extract data
Extractor	Detection Engine	Check tampering
Detection Engine	CSV Manager	Store result
Dashboard	CSV Manager	Fetch data

User Interface

- Upload page
- Dashboard page

Configuration Interface

- Flask config variables:
  - UPLOAD_FOLDER
  - MAX_CONTENT_LENGTH

5. State and Session Management

Component	State	Storage
Upload Process	Temporary files	File system
Results	Analysis data	CSV
Session	Flash messages	Flask session

6. Caching

- No caching implemented
- System processes data in real-time

7. Non-Functional Requirements

7.1 Security Aspects

- Secure filename handling (prevents path traversal)
- File size limit (50MB)
- No external exposure

7.2 Performance Aspects

- Optimized for small datasets

- Fast execution using local processing

- No heavy computations or external API calls

8. References

1.**Flask Documentation**

Official documentation for Flask framework used in backend development

https://flask.palletsprojects.com/

2.**Python os Module Documentation**

Used for file metadata extraction (ctime, mtime, file size)

https://docs.python.org/3/library/os.html